CSE 221: Algorithms

Chapter 5 — Divide-and-Conquer

Prepared by Ariyan Hossain [AYO]

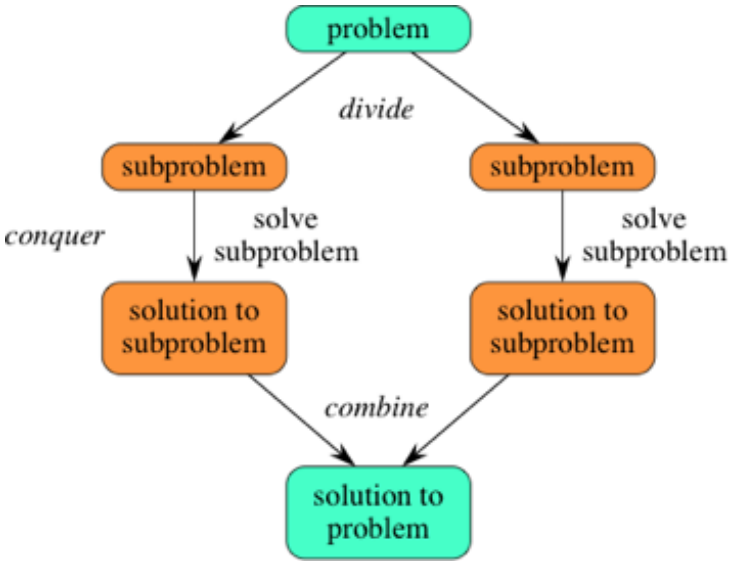
1 Divide-and-Conquer

One of the most powerful techniques for solving problems is to break them down into smaller, more easily solved pieces. Smaller problems are less overwhelming, and they permit us to focus on details that are lost when we are studying the entire problem. A **divide-and-conquer** algorithm solves a problem recursively. If the problem is small enough (**base case**), we solve it directly. Otherwise (**recursive case**), we do:

Steps of Divide-and-Conquer

- **Divide:** split the problem into one or more smaller subproblems of the same type.
- **Conquer:** solve the subproblems recursively.
- **Combine:** combine subproblem solutions to solve the original problem.

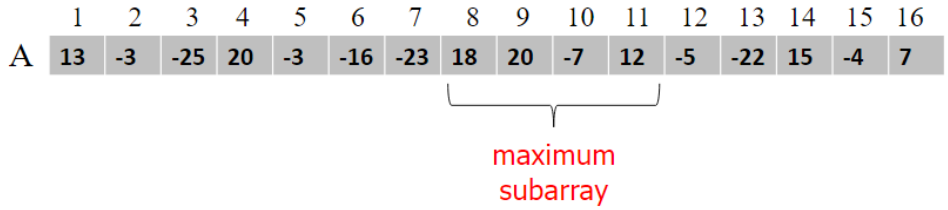
Divide-and-conquer is not a single algorithm you "apply" to every task; it is a way of thinking about designing algorithms.



Steps of Divide-and-Conquer

2 Maximum Subarray Problem

Input: an array $A[1..n]$ of n numbers (numbers may be negative).
Output: a non-empty subarray $A[i..j]$ having the largest sum



Location of maximum subarray from an array

2.1 Brute force idea

Try all possible contiguous subarrays and compute their sums; pick the maximum.

$$\begin{array}{cccccc} A[1..1], & A[1..2], & A[1..3], & \cdots & A[1..(n-1)], & A[1..n] \\ & A[2..2], & A[2..3], & \cdots & A[2..(n-1)], & A[2..n] \\ & \vdots & \vdots & & \vdots & \vdots \\ & & & & A[(n-1)..(n-1)], & A[(n-1)..n] \\ & & & & & A[n..n] \end{array}$$

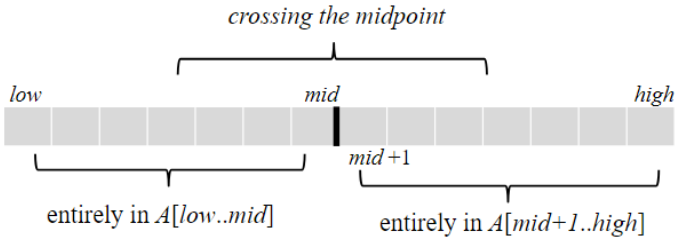
Maximum Subarray on array A using Brute Force

```
MAX-SUBARRAY-BRUTEFORCE(A)
    best ← -infinity
    for i ← 1 to n
        sum ← 0
        for j ← i to n
            sum ← sum + A[j]
            best ← max(best, sum)
    return best
```

The brute force method uses two nested loops, so it takes $\mathcal{O}(n^2)$ time.

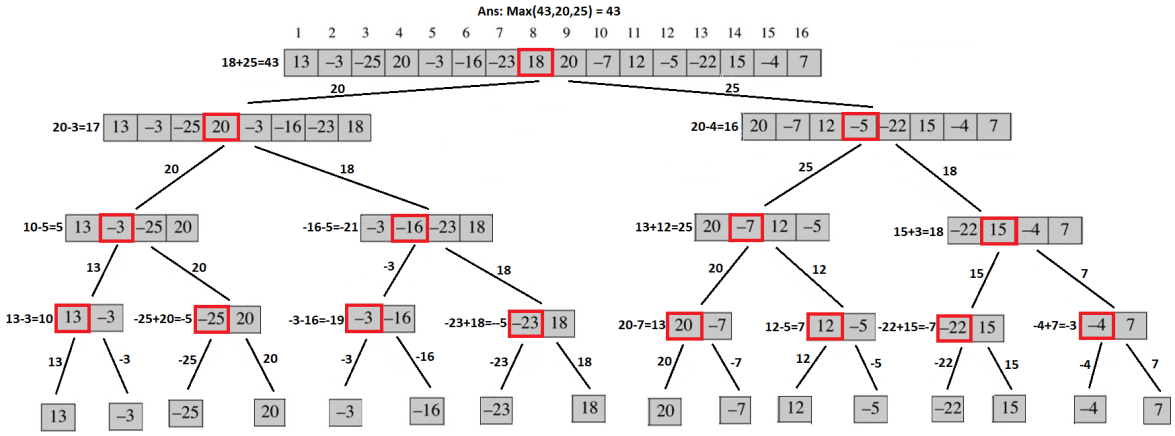
2.2 Divide-and-Conquer solution

1. Divide the given array into two halves.
2. Return the max of the following three:
   (a) Recursively calculate max subarray in left half
   (b) Recursively calculate max subarray in right half
   (c) Recursively calculate max subarray sum such that the subarray crosses midpoint
      i. Find the maximum sum starting from mid point and ending at some point on left of mid
      ii. Find the maximum sum starting from mid+1 point and ending at some point on right of mid+1
      iii. Finally, combine the two and return



Possible locations of the maximum subarray

2.2.1 Simulation (Maximum Subarray)



Simulation of maximum subarray with left, right and cross sum

2.2.2 Cross-Sum Operation

Let,

$$A = [20,\ -7,\ 12,\ -5,\ -22,\ 15,\ -4,\ 2], \quad low = 1,\ high = 8,\ mid = 4$$

20	−7	12	−5	−22	15	−4	2

split between mid=4 and mid+1=5

Left scan possibilities (from mid to low):

L_4 : $\boxed{-5}$ $\Rightarrow -5$

L_3 : $\boxed{12}\ \boxed{-5}$ $\Rightarrow 12 + (-5) = 7$

L_2 : $\boxed{-7}\ \boxed{12}\ \boxed{-5}$ $\Rightarrow -7 + 12 + (-5) = 0$

L_1 : $\boxed{20}\ \boxed{-7}\ \boxed{12}\ \boxed{-5}$ $\Rightarrow 20 + (-7) + 12 + (-5) = 20$

Right scan possibilities (from $mid + 1$ to $high$):

R_5 : $\boxed{-22}$ $\Rightarrow -22$

R_6 : $\boxed{-22}\ \boxed{15}$ $\Rightarrow -22 + 15 = -7$

R_7 : $\boxed{-22}\ \boxed{15}\ \boxed{-4}$ $\Rightarrow -22 + 15 + (-4) = -11$

R_8 : $\boxed{-22}\ \boxed{15}\ \boxed{-4}\ \boxed{2}$ $\Rightarrow -22 + 15 + (-4) + 2 = -9$

Best choices: $\max(L) = 20$ (from L_1), $\max(R) = -7$ (from R_6)

Crossing subarray: $\boxed{20}\ \boxed{-7}\ \boxed{12}\ \boxed{-5}\ \boxed{-22}\ \boxed{15}$ $\Rightarrow$ Cross-Sum $= 20 + (-7) = 13$

2.2.3 Pseudocode (Maximum Subarray)

Maximum Subarray of array A using Divide & Conquer

```
MAXIMUM-SUBARRAY(A, low, high)
    if high == low
        return A[low]          // base case: only one element
    else
        mid ← (low + high) // 2

        max_left_sum  ← MAXIMUM-SUBARRAY(A, low, mid)
        max_right_sum ← MAXIMUM-SUBARRAY(A, mid + 1, high)
        max_cross_sum ← MAX-CROSS-SUBARRAY(A, low, mid, high)

        return max(max_left_sum, max_right_sum, max_cross_sum)

MAX-CROSS-SUBARRAY(A, low, mid, high)
    left_sum ← -infinity       // max subarray of form A[i..mid]
    sum ← 0
    for i ← mid down to low
        sum ← sum + A[i]
        if sum > left_sum
            left_sum ← sum
            max_left ← i

    right_sum ← -infinity      // max subarray of form A[mid+1..j]
    sum ← 0
    for j ← mid + 1 to high
        sum ← sum + A[j]
        if sum > right_sum
            right_sum ← sum
            max_right ← j

    return left_sum + right_sum
```

Follow-up Questions

Q) What is the time complexity of Cross-Sum operation?
Ans: $\mathcal{O}(\frac{n}{2} + \frac{n}{2}) = \mathcal{O}(n)$

Q) What is the recurrence equation and time complexity of Maximum Subarray?
Ans: It depends on the number of subproblems, each subproblem size, and the work done at each step. In each step, there are 2 subproblem2 where size gets divided by 2 (n/2) and work done is n at each step.

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &= \mathcal{O}(n \log_2 n) \end{aligned}$$

3 Karatsuba's Multiplication Algorithm

Input: two n-digit numbers x and y.
Output: the product $x \cdot y$.

3.1 Brute force multiplication idea

Grade-school multiplication performs n digit-multiplications for each of the n digits, so it takes $\mathcal{O}(n^2)$ time.

				1	2	3	4		$\rightarrow x$
			$\times$	8	7	6	5		$\rightarrow y$
				6	1	7	0	$\rightarrow$	n operations
			7	4	0	4		$\rightarrow$	n operations
		8	6	3	8			$\rightarrow$	n operations
	9	8	7	2				$\rightarrow$	n operations
1	0	8	1	6	0	1	0		

3.2 Divide-and-conquer idea

Write the numbers by splitting them into halves, where m is half the number of digits:

$$x = a \cdot 10^m + b, \qquad y = c \cdot 10^m + d$$

Then,

$$xy = (a \cdot 10^m + b)(c \cdot 10^m + d) = ac \cdot 10^{2m} + (ad + bc) \cdot 10^m + bd$$

Karatsuba's key trick computes $(ad + bc)$ using one multiplication:

$$ad + bc = (a + b)(c + d) - ac - bd$$

Since ac and bd are already computed, we only need one additional multiplication to compute $(a+b)(c+d)$. Thus, instead of performing four multiplications on $n/2$-digit numbers, Karatsuba performs only three.

$$xy = ac \cdot 10^{2m} + ((a + b)(c + d) - ac - bd) \cdot 10^m + bd$$

Example Calculation of Karatsuba's Multiplication

Let

$$x = 146123, \quad y = 352120 \quad (m = 3)$$

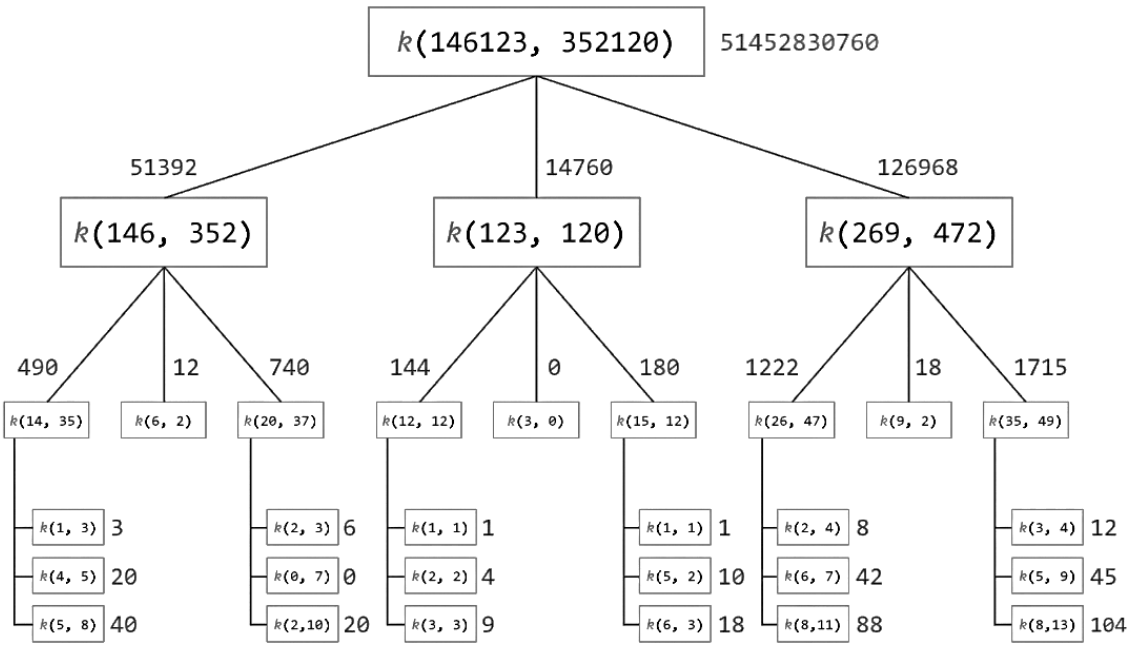
$$x = 146\,(a) \cdot 10^3 + 123\,(b), \quad y = 352\,(c) \cdot 10^3 + 120\,(d)$$

$$ac = 146 \times 352 = 51392, \quad bd = 123 \times 120 = 14760 \text{ (Recursively)}$$

$$(a + b)(c + d) = (146 + 123)(352 + 120) = 269 \times 472 = 126968 \text{ (Recursively)}$$

$$xy = ac \cdot 10^{2m} + ((a + b)(c + d) - ac - bd) \cdot 10^m + bd = 51452830760$$

3.2.1 Simulation (Karatsuba's Multiplication)



3.2.2 Pseudocode (Karatsuba's Multiplication)

Karatsuba Multiplication of integers x and y using Divide & Conquer

```
KARATSUBA(x, y)
    if x < 10 or y < 10
        return x * y      // base case (single digit)

    n   ← max(digits(x), digits(y))
    mid ← n // 2

    a ← x // 10^mid      // first half of a
    b ← x % 10^mid       // second half of a
    c ← y // 10^mid      // first half of b
    d ← y % 10^mid       // second half of b

    S1 ← KARATSUBA(a, c)            // ac
    S2 ← KARATSUBA(b, d)            // bd
    S3 ← KARATSUBA(a + b, c + d)    // (a+b)(c+d)
    S4 ← S3 - S2 - S1               // ad + bc

    return S1 * 10^(2*mid) + S4 * 10^mid + S2
```

Follow-up Questions

Q) What would happen if Karatsuba did not apply the trick and use 4 multiplications??
Ans: Then the recurrence would be $T(n) = 4T(\frac{n}{2}) + n = \mathcal{O}\left(n^{\log_2 4}\right) = \mathcal{O}(n^2)$, which is the same as the running time of the brute-force method.

Q) What is the recurrence equation and time complexity of Karatsuba's Multiplication Algorithm?
Ans: It depends on the number of subproblems, each subproblem size, and the work done at each step. In each step, there are 3 subproblems where size gets divided by 2 (n/2) and work done is n at each step due to adding or subtracting 2 n-bit numbers.

$$\begin{aligned} T(n) &= 3T\left(\frac{n}{2}\right) + n \\ &= \mathcal{O}(n^{\log_2 3}) \\ &= \mathcal{O}(n^{1.58}) \end{aligned}$$

Q) How to apply the algorithm when two numbers are not equal in length?
Ans: Pad the shorter number with leading zeros until both lengths are equal.

Q) Can we use this on other number-based representations such as binary?
Ans: Yes, just replace base 10 with base 2 in the calculations.